

# Project Design Document: The Campus Exchange

## Comprehensive Feature Specification & Implementation Logic

*Target Platform: Android (Kotlin/Compose) — Backend: Firebase*

### 1. The Mining Engine (Capital Generation)

---

This module handles the "Proof of Work" (PoW) required to inject currency into the ecosystem.

#### 1.1. Hardware-Backed Pedometer

- **Implementation:** Utilize `Sensor.TYPE_STEP_COUNTER`. Unlike the accelerometer-based step detector, this returns the total steps since the last reboot, allowing for lower battery consumption.
- **The Claim Ritual:** Steps are stored in a "Pending" buffer. Users must manually trigger a `claimSteps()` function to convert them to **Campus Coins**. This reinforces the psychological value of "earned" capital.

#### 1.2. Anti-Cheat & Sensor Fusion

- **Gait Analysis:** Cross-reference step frequency with `Sensor.TYPE_ACCELEROMETER`. If the movement is a perfectly rhythmic sine wave (indicating a mechanical phone shaker), the mining efficiency drops to zero.
- **GPS Displacement:** A minimum displacement requirement (e.g., 10 meters per 100 steps) via `FusedLocationProvider` ensures steps are taken during actual transit rather than stationary shaking.

#### 1.3. Multiplier Zones & Focus Mining

- **Geofenced Boosts:** Define Lat/Lng polygons in Firestore. When a user is inside a "Gym Zone," the conversion rate shifts from 100:1 to 50:1.
- **Focus Mining:** Utilizing `Sensor.TYPE_LIGHT` and `Sensor.TYPE_PROXIMITY`, users earn "Focus Coins" if the device is stationary and face-down for > 20 minutes, incentivizing library study sessions.

### 2. Simulated Stock Market (The Brokerage)

---

A high-frequency trading simulation using a localized Limit Order Book (LOB).

### 2.1. The Matching Engine

- **Order Types:** Support for **Market Orders** (instant execution at best price) and **Limit Orders** (execution only at price  $P$  or better).
- **The Order Book:** A Firestore collection of **Bids** and **Asks**. A Cloud Function triggers on every new entry to check for price overlaps and execute trades.

### 2.2. Automated Market Maker (AMM) Fallback

- **Liquidity Provision:** To prevent "dead markets," the system implements a constant product formula:  $x \times y = k$ . If no human sellers exist for "Hostel A" shares, the app calculates a price based on the ratio of coins to shares in the "Campus Treasury" pool.

### 2.3. Market News & Volatility

- **Event-Driven Pricing:** Integration of a "News Feed." Admins post updates (e.g., "Mess Menu: Special Dinner Tonight"). This triggers a script to simulate a "buy wall" for Canteen stocks, teaching users about event-driven trading.

## 3. Parimutuel Staking (Social Betting)

---

A risk-management layer where users stake capital on campus-wide binary or multi-choice outcomes.

### 3.1. Live Odds Calculation

- **The Formula:** The potential payout for a winning stake of amount  $s$  is:  $Payout = s + (s/W) \times L \times (1 - f)$ , where  $W$  is total winning stakes,  $L$  is total losing stakes, and  $f$  is the house fee.
- **Real-time UI:** The "Potential Win" amount must update every time the Firestore `pool_total` changes.

### 3.2. The Burn Treasury

- **Inflation Control:** A 5% fee from every pool is sent to a `null` address (Burned) or the `CampusTreasury` to prevent the "walking" economy from devaluing the coin.

## 4. Gamification & Social Layers

---

### 4.1. Dynamic Title System (The Decay Logic)

Titles are not permanent milestones; they are reflects of *recent* velocity.

- **The Algorithm:**  $Reputation_t = (Reputation_{t-1} \times 0.8) + (Activity_{today})$ .
- **Tiers:**
  - **Grandmaster:** Top 1% in net worth + active trade in last 48h.
  - **Maverick:** > 20% portfolio growth in 24h.
  - **Apex Predator:** > 20,000 steps for 3 consecutive days.

## 4.2. Hostel Wars & Airdrops

- **Group Mining:** Aggregated step counts for Hostels. The winning Hostel each week receives a "Liquidity Airdrop" distributed to all active residents.

## 5. Infrastructure & Security

---

### 5.1. Atomic Transactions

- **Concurrency Control:** All trades must use `Firestore.runTransaction()`. This prevents a user from spending coins on a stock and a bet at the exact same millisecond, ensuring the wallet balance never drops below zero.

### 5.2. Admin Command Center

- **Market Circuit Breakers:** A "Kill Switch" to halt trading on specific assets during high-volatility bugs.
- **Oracle Resolution:** A secure UI for authorized admins to finalize pool results (e.g., entering the final score of a match).